

Tutorial: Simple Power Analysis with ChipWhisperer Lite (Password)

Ryan Evans, Logan Reichling, and Boyang Wang

Data and System Security Lab (DaSec), ECE Department, University of Cincinnati

Last modified: August 2026, Status: Completed

1 Overview

This document provides and explains the setup and workflow required in order to perform simple power analysis on a software implementation of a password checker program using ChipWhisperer Lite. The simple power analysis, if successful, can reveal the correct password hard-coded in a program by analyzing the power trace from the execution of a program. Simple power analysis is an introductory type of side-channel analysis, which infers secret/sensitive information based on power consumption.

This document is primarily based on the side-channel analysis notebook¹ from ChipWhisperer website. We made some minor customized changes.

Acknowledgments. The research and education efforts associated with this document are supported by CHEST, the NSF IUCRC Center for Hardware and Embedded Systems Security and Trust.

2 Software and Hardware Settings

In this document, we use the following hardware and software.

- Hardware: ChipWhisperer Lite with an ARM STM32F3 microcontroller as a target, USB cable (both available from CW Lite).
- Operating System: Ubuntu 22.04 (or a VM with Ubuntu 22.04), several Linux packages, Python packages, and our customized jupyter notebook on GitHub repository ²

Note: Skip Section 3 and Section 4.1 if we have done it in other CW Lite tutorials.

3 ChipWhisperer Environment Setup

3.1 ChipWhisperer Overview

ChipWhisperer is an open source toolchain dedicated to hardware security research. This toolchain consists of several layers of open source components, including hardware, firmware, and

¹ [https://github.com/newaetech/chipwhisperer/jupyter/courses/sca101/Lab_2_1B - Power Analysis for Password Bypass \(HARDWARE\).ipynb](https://github.com/newaetech/chipwhisperer/jupyter/courses/sca101/Lab_2_1B_-_Power_Analysis_for_Password_Bypass_(HARDWARE).ipynb)

² https://github.com/UCdasec/Tutorial-CW/main/Setup_Scripts/Setup_CW_Lite_Fault_Password.ipynb

software. More information regarding ChipWhisperer can be found on ChipWhisperer's GitHub repository³ and official website⁴.

There are multiple ChipWhisperer hardware available on the market. In this tutorial document, we will use a ChipWhisperer Lite (with an ARM STM32F3 microcontroller as a target). An picture of a CW Lite (with an AVR XMEGA microcontroller as a target) from ChipWhisperer website⁵ is presented in Fig. 1. The CW main board (on the left side of the picture) is in charge of communicating with the target and capturing power traces from the target board. The target board (on the right side of the picture) is where we can upload a binary (e.g., a binary of AES encryption program) and execute it on the target microcontroller.

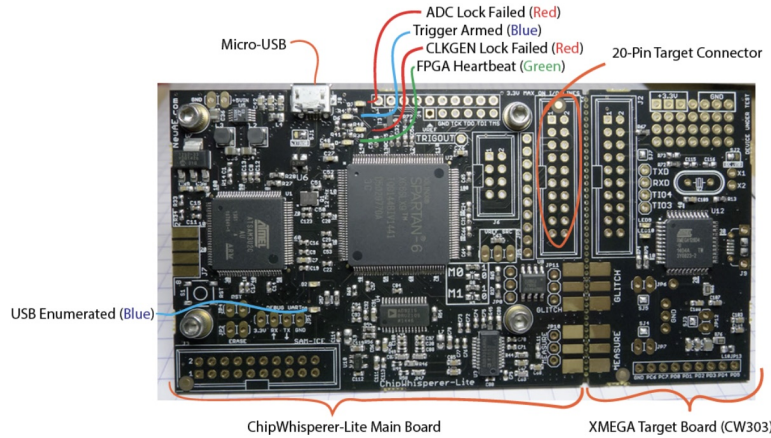


Fig. 1: CW Lite with an AVR XMEGA microcontroller as a target

ChipWhisperer Lite (with an ARM STM32F3 microcontroller as a target) shares the same main board and same layout of the target board but with an ARM STM32F3 microcontroller as a target. A picture of it can be found below.

3.2 ChipWhisperer Installation

Before we connect CW Lite hardware to our computer, we first need to install the ChipWhisperer environment. Specifically, we can follow the instructions and steps from this ChipWhisperer webpage⁶ to install the ChipWhisperer environment. We assume that we are using a computer running Ubuntu or a VM running Ubuntu. We can either choose (1) *the quick installation option* by running all the commands at once or *the manual installation option* by running these commands step by step mentioned on the webpage. We recommend the manual installation option. Specifically,

³ <https://github.com/newaetech/chipwhisperer>

⁴ <https://chipwhisperer.readthedocs.io/en/latest/getting-started.html>

⁵ <https://chipwhisperer.readthedocs.io/en/latest/getting-started.html>

⁶ <https://chipwhisperer.readthedocs.io/en/latest/linux-install.html>

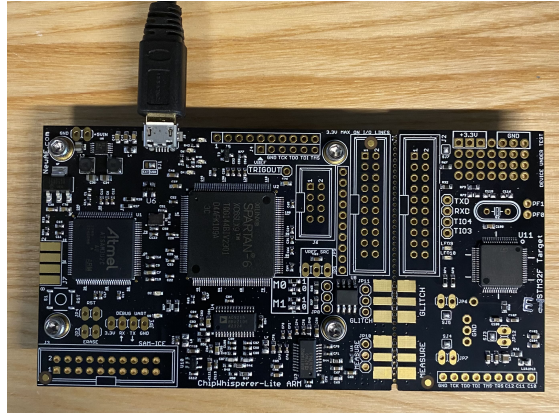


Fig. 2: CW Lite with an ARM STM32F3 microcontroller as a target

we can start from the block below and continue all the remaining steps until we reach the end of the webpage. The entire process takes **8~15 minutes** depending on the system. We also provide a short video⁷ walking through each step during the manual installation.

- Introduction
- Getting Started
- Overview
- Installation
- Windows Installation
- Windows Drivers
- Linux Installation
- Mac OS X Installation
- Virtual Machine Installation
- Running ChipWhisperer
- Updating ChipWhisperer
- Support

Manual Installation

Required Packages

Begin by updating all your packages:

```
sudo apt update && sudo apt upgrade
```

Next, grab the prerequisites for building firmware for targets, as well as python:

```
sudo apt install libusb-dev make git avr-libc gcc-avr \
gcc-arm-none-eabi libusb-1.0-0-dev usbutils
```

Fig. 3: Manual installation for ChipWhisperer (from ChipWhisperer website)

Make sure we reboot the system as instructed during the process. At the end of the installation, we should be able to enable the ChipWhisperer environment and run jupyter notebook successfully. Specifically, when we run

```
1 source ~/.cwvenv/bin/activate
```

We will see the “**(.cwvenv)**” to the left of working directory if the environment is installed successfully

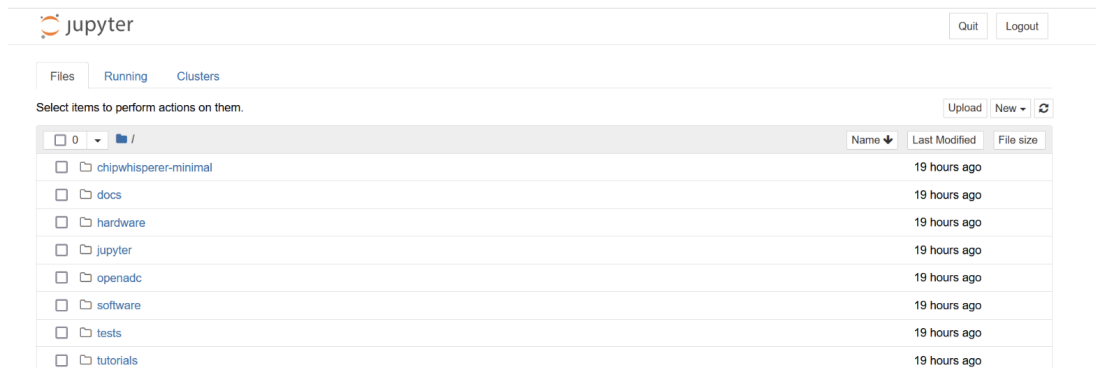
```
dung-dang@dungdang-ThinkPad-T15p-Gen-3:~$ source ~/.cwvenv/bin/activate
(.cwvenv) dung-dang@dungdang-ThinkPad-T15p-Gen-3:~$
```

⁷ https://ceas.mediaspace.kaltura.com/media/Tutorial_CW_Installation/1_zytbs228

Next, we can open the jupyter lab by running the command below in the terminal

```
1 jupyter lab
```

The Jupyter server will begin to boot. Wait until the Jupyter lab automatically opens in the web browser and we should be able to see similar as shown below.



When we start Jupyter Lab, it runs as a foreground process and takes over that terminal window to keep the server alive. This blocks the current terminal session, preventing us from executing additional commands in that specific window. If we need to run other commands, we can open a new terminal tab or press “Ctrl” + “C” then press “Y” to exit the Jupyter lab.

3.3 Connect and Verify Hardware

Now, we can connect ChipWhisperer Lite to our computer by using the USB cable (offered by CW Lite box). The computer should be able to automatically recognize the CW Lite. If the connection is successful, we should observe the blue light on the board is ON and flashes (as shown in Fig. 7).

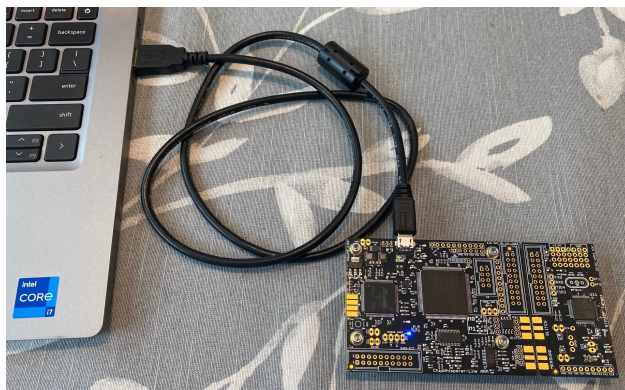


Fig. 4: CW Lite is connected to a computer (blue light flashes)



Fig. 5: Dropdown menu of VM

If we use a VM, make sure we attach the ChipWhisperer USB to the VM through the USB dropdown (as shown in Fig. 5)

Next, run the following command to enable ChipWhisperer environment

```
1 source ~/.cwvenv/bin/activate
2 jupyter lab
```

In Jupyter notebook window, click and choose `chipwhisperer` ⇒ `jupyter` ⇒ `Setup_Scripts`, and open `Setup_Generic.ipynb`.

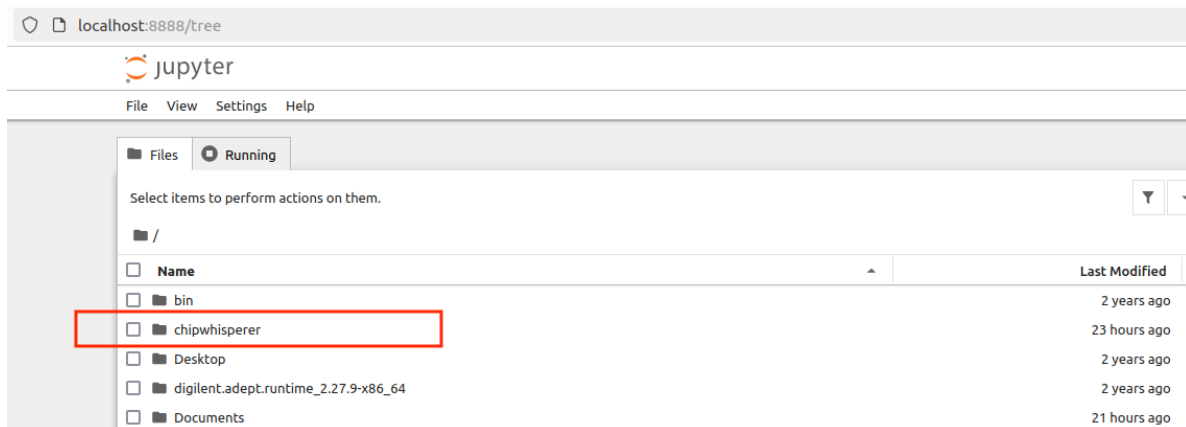


Fig. 6: Jupyter notebook (`./chipwhisperer`)

Execute the first code block of `Setup_Generic.ipynb` by clicking “Run” as shown in Fig. 8. After 1 or 2 seconds, we should observe it outputs `INFO: Found ChipWhisperer`, which indicates our computer recognize the CW Lite hardware successfully. At the same time, we should also observe that an additional green light is ON and flashes on the board. We also provide a short video⁸ walking through each step during this hardware connection testing process.

⁸ https://ceas.mediaspace.kaltura.com/media/Tutorial_CW_Connection_Verify/1_800go4yr

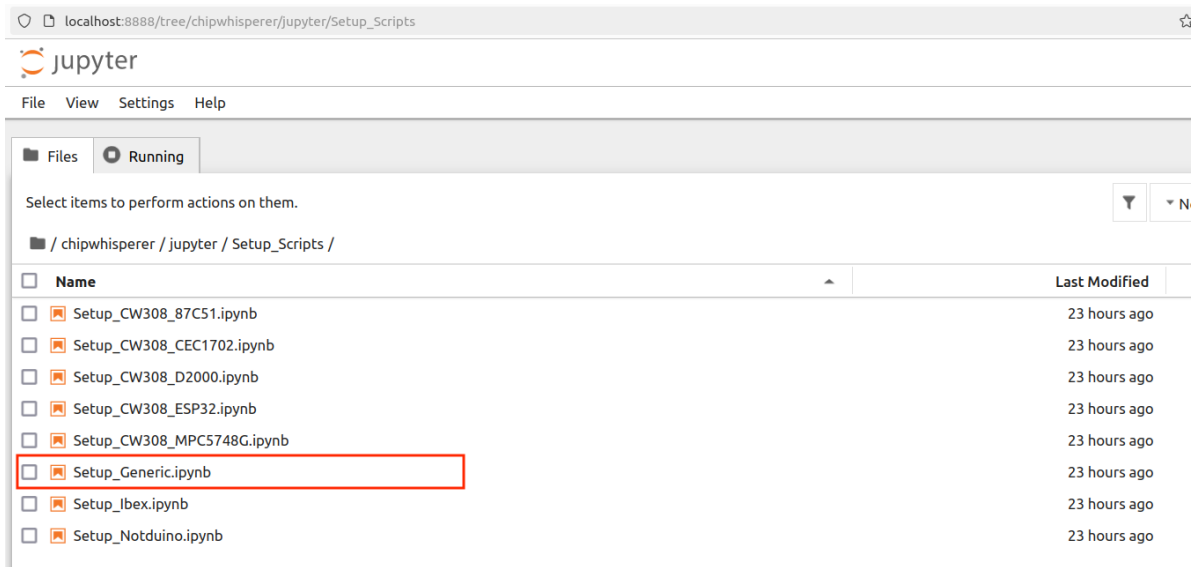


Fig. 7: Jupyter notebook (./chipwhisperer/jupyter/Setup_Scripts)



Fig. 8: Run the first code block of Setup_Generic.ipynb

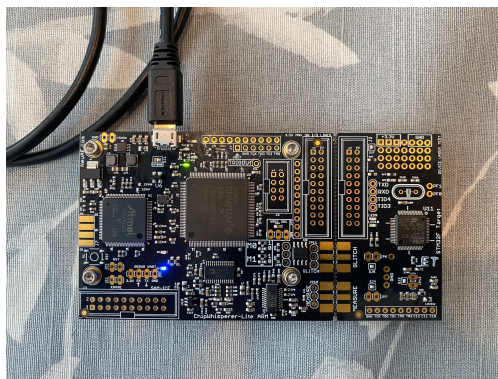


Fig. 9: CW Lite is recognized by the computer (green light flashes)

4 Simple Power Analysis on Password Checker Program

4.1 Install Additional Libraries and Clone Our Repository

Assuming we have installed ChipWhisperer environment, connect and verify the connection to CW Lite hardware successfully as described above, we now install a few additional libraries needed for our customized jupyter notebook and download it.

- Close and exit the previous jupyter sessions (Ctrl + C)
- Ensure that CW Lite hardware is connected
- Enable the CW environment is enabled by running the following command again

```
1 source ~/.cwenv/bin/activate
```

- Install the following Python packages with the following command. These packages are needed for our customized jupyter notebook.

```
1 pip install ipython scikit-learn PyCryptodomex tqdm reportlab h5py
```

- Ensure we are working in the home directory by typing

```
1 cd ~/
```

- Clone our GitHub repository⁹ by running the following command

```
1 git clone https://github.com/UCdasec/Tutorial-CW.git
```

4.2 Password Checker Program

We use a password checker program, which is part of the C code `basic-passwdcheck.c` provided by ChipWhisperer. If we have installed chipwhisperer already in `/home/username`, a copy of this implementation can be found here¹⁰. We also present the program below

```
1 // basic-passwdcheck.c
2 // .....
3 int main(void) {
4     platform_init();
5     init_uart();
6     trigger_setup();
7
8     char passwd[32];
9     char correct_passwd[] = "?????"; // the actual password is hidden
10     intentionally
11
12     while(1) {
```

⁹ <https://github.com/UCdasec/Tutorial-CW>

¹⁰ `home/username/chipwhisperer/firmware/mcu/basic-passwdcheck/basic-passwdcheck.c`

```

12     my_puts("*****Safe-o-matic_3000_Booting...\n");
13     my_puts("Decrypting_database..[DONE]\n");
14     delay_2_ms();
15     my_puts("\n\n");
16     //Give them one last warning
17     my_puts("WARNING:_UNAUTHORIZED_ACCESS_WILL_BE_PUNISHED\n");
18
19     trigger_low();
20
21     //Get password
22     my_puts("Please_enter_password_to_continue:_");
23     my_read(passwd, 32);
24     uint8_t passbad = 0;
25
26     trigger_high();
27
28     for(uint8_t i = 0; i < sizeof(correct_passwd); i++){
29         if (correct_passwd[i] != passwd[i]){
30             passbad = 1;
31             break;
32         }
33     }
34
35     if (passbad){
36         // Stop them fancy timing attacks
37         int wait = 1;
38         for(volatile int i = 0; i < wait; i++){
39             ;
40         }
41         delay_2_ms();
42         delay_2_ms();
43         my_puts("PASSWORD_FAIL\n");
44         led_error(1);
45     } else {
46         my_puts("Access_granted,_Welcome!\n");
47         led_ok(1);
48     }
49
50     //All done;
51     while(1);
52 }

```

As we can see from the above program, the real password (?????) is hard-coded in the program for the ease of demonstration. Note that we intentionally hide the real password for now. However, we know that it has 5 characters in total and each character is one of the characters from a-z or 0-9.

If the given password from a user is correct, the variable `passbad` remains to be 0. Otherwise, `passbad` will be updated to 1. ChipWhisperer’s simple serial protocol will retrieve/read the value of `passbad` and send it back to the computer connect to CW Lite by running `my_puts()` function.

The logic of this password checker program is that it verifies the password one character after another. Specifically, if the first character is correct, then it will move on checking the next character. Otherwise, it will break and quite the program right away. This program behavior offers a great opportunity to decide whether a character is correct or not as all the power traces from the first incorrect characters will share the same/similar pattern while *the one from the first correct character will be distinguishable from others*.

In this tutorial, we will leverage this observation to decide the first character of the hard-coded password by analyzing power traces. The process can be repeated and expanded easily to recover every character in the password.

We can also modify or customize the C code of this password checker program if necessary. In this tutorial, we use the default one from ChipWhisperer without any modifications.

4.3 Run Power Trace Collection (Password)

To collect power traces on this password checker program on ChipWhisperer Lite, we can follow the steps below. In addition, we also provide a short video¹¹ walking through each step for this simple power analysis.

1. Open Terminal and run the command below to enable the environment to enable the Jupyter Notebook if needed

```
1 source ~/.cwenv/bin/activate
2 jupyter lab
```

A web browser tab for jupyter notebook will open.

2. Connect ChipWhisperer Lite to our computer and ensure only one blue light is ON and flashes (if not, gently disconnect the USB cable and reconnect). Otherwise, we may get `scope() not defined` error later on.
3. In the web browser for Jupyter notebook, direct to “/Tutorial-CW/Setup_Scripts”, then click on the “**Setup_CW_Lite_SCA_Password.ipynb**”. This script contains multiple code cells to initialize the settings, compile and load a binary to the target board, and collect power traces of the binary while it is running on the target board. We will explain each cell one by one below.
4. Before we run any cells in this notebook, we can restart the kernel and clean all the outputs from previous runs.

¹¹ https://ceas.mediaspace.kaltura.com/media/Tutorial_CW_Lite_SCA_Password/1_s0qiutic

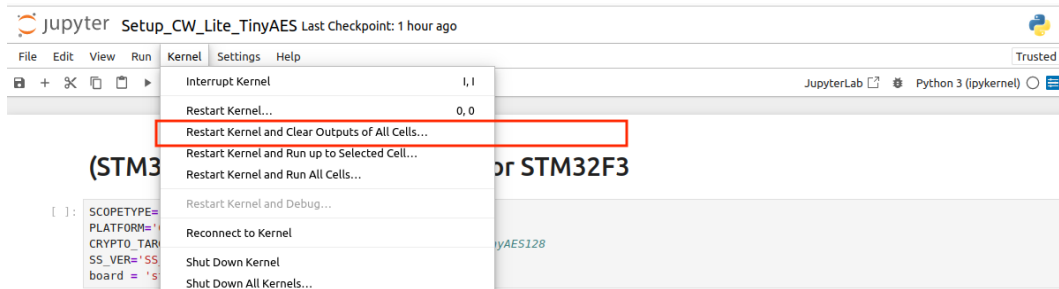


Fig. 10: Restart kernel and clean all the outputs

- Run the first cell to enable STM32F3 as the target board. Make sure the SCOPETYPE is OPENADC and PLATFORM is CWLITEARM. To run an individual cell, select the cell by left-clicking on it, then press the “Run” button

(STM32F) Setup for STM32F3

First you'll need to select which hardware setup you have. You'll need to select both a 'SCOPETYPE' and a 'PLATFORM'. SCOPETYPE can either be 'OPENADC' for the CWLite/CW1200 or 'CWNANO' for the CWNano. PLATFORM is the target device, with 'CWLITEARM' / 'CW308_STM32F3' being the best supported option, followed by 'CWLITEXMEGA' / 'CW308_XMEGA', then by 'CWNANO'. As of CW 5.4, you can select the SimpleSerial version used. For example:

```
SCOPETYPE = 'OPENADC'
PLATFORM = 'CWLITEARM'
SS_VER = 'SS_VER_2_1'
```

```
SCOPETYPE = 'OPENADC'
PLATFORM = 'CWLITEARM'
SS_VER = 'SS_VER_2_1'
```

Fig. 11: Choose the target board setting (Run the STM32F one only)

- Next, we can run the **first cell** from “Detect, Compile, and Upload to ChipWhisperer” to detect the CW Lite hardware. If successful, we should see the message INFO: Found ChipWhisperer again along with some scope information. Make sure we update the username ('boyang') in the path to the one in our system.

Detect, Compile, and Upload to ChipWhisperer

This code will connect the scope and do some basic setup. We're now just going to use a special setup script to do this. This script contains the commands we ran separately before. Update username 'boyang' to your username or update the path accordingly for your system.

```
%run "/home/boyang/chipwhisperer/jupyter/Setup_Scripts/Setup_Generic.ipynb"
```

```
INFO: Found ChipWhisperer
scope.gain.mode          changed from low          to high
scope.gain.gain          changed from 0           to 30
scope.gain.db            changed from 5.5         to 24.8359375
scope.adc.basic_mode     changed from low         to rising_edge
scope.adc.samples        changed from 24400       to 5000
```

Fig. 12: Detect CW Lite

7. Next, we run the **second cell** from “Detect, Compile, and Upload to ChipWhisperer” to compile the C code of password checker program. We should observe the binary of the program, named (basic-passwordcheck-CWLITEARM.elf), is generated successfully.

The following code will build the firmware for the target. Update username 'boyang' to your username or update the path accordingly for your system.

```
%$bash -s "$PLATFORM" "$SS_VER"
cd /home/boyang/chipwhisperer/firmware/mcu/basic-passwordcheck
make PLATFORM=$1 CRYPTO_TARGET=NONE SS_VER=$2 -j

SS_VER set to SS_VER_2_1
SS_VER set to SS_VER_2_1
arm-none-eabi-gcc (15:10.3-2021.07-4) 10.3.1 20210621 (release)
Copyright (C) 2020 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

mkdir -p objdir-CWLITEARM
```

Fig. 13: Compile the C code of Password Checker

```
LINKING:
-en basic-passwordcheck-CWLITEARM.elf ...
Memory region      Used Size  Region Size  %age Used
      RAM:         1176 B       40 KB       2.87%
      ROM:         4812 B       256 KB       1.84%
-e Done!
.
.
Creating load file for Flash: basic-passwordcheck-CWLITEARM.hex
Creating load file for Flash: basic-passwordcheck-CWLITEARM.bin
```

Fig. 14: Compile the C code of Password Checker (Expected Output)

8. Next, we run the **third cell** from “Detect, Compile, and Upload to ChipWhisperer” to flash the binary to the target board. We should see the message saying “Verified flask OK” for the binary file in hex format. Make sure we update the username in the file path based on our system setting.

Finally, all that's left is to program the device, which can be done with the following line: Update username 'boyang' to your username or update the path accordingly for your system.

```
cw.program_target(scope, prog, "/home/boyang/chipwhisperer/firmware/mcu/basic-passwordcheck/basic-pass

Detected known STM32F32: STM32F302xB(C)/303xB(C)
Extended erase (0x44), this can take ten seconds or more
Attempting to program 4811 bytes at 0x8000000
STM32F Programming flash...
STM32F Reading flash...
Verified flash OK, 4811 bytes
```

Fig. 15: Flash the binary to target board (Expected output)

9. Next, we can review the source C code again for this program.
10. we run the next cell (“Setup Trace Collection Function”) to setup the power trace collection function `cap_pass_trace(pass_guess)`, where we can pass a password `pass_guess` to it later on. Inside the function, the trace collection is enabled by running `scope.arm()` and

`scope.capture()`. Function `response = target.read(num_char, 10)` helps us read the response back from the target and show the associated information print out by the program by running `print(response)`.

Setup Trace Collection Function

To make interacting with the hardware easier, let's define a function to attempt a password and return a power trace:

```
def cap_pass_trace(pass_guess):
    reset_target(scope)
    num_char = target.in_waiting()
    while num_char > 0:
        response = target.read(num_char, 10)
        time.sleep(0.01)
        num_char = target.in_waiting()
        print(num_char)
        print(response)

    scope.arm()
    target.write(pass_guess)
    ret = scope.capture()
```

Fig. 16: Define a trace collection function

11. Next, we run the next cell to define the number of samples per trace we would like to have. By default, CW Lite can support up to 5,000 samples per trace. In this example, 150 or 200 is sufficient to observe the power pattern for the operation associated with first character. Let us assume we choose 150 in this remaining example.

We also don't need all of the default 5000 samples in the trace. 150 is a good starting point for the executions associated with the verification on the first character of this target password program:

```
scope.adc.samples = 150
```

Fig. 17: Define the number of samples per trace as 150

12. Next, we run the next cell, which sends a password with a single character 'h' to the target board and generates the plot of the collected power trace.

Sending a password with a single char 'h'

```
trace_test1 = cap_pass_trace("\n")

#Basic sanity check
assert(len(trace_test1) == 150)
print("✓ OK to continue!")

cw.plot(trace_test1).opts(color='blue')

0
PASSWORD FAIL
*****Safe-o-matic 3000 Booting...
Decrypting database..[DONE]
```

Fig. 18: Sending a password with a single char 'h'

We should expect the program indicates that the password is incorrect and says ‘PASSWORD FAIL’ from the terminal. We should also obtain a power trace associated with the execution of the program given this password as shown in Fig. 19.

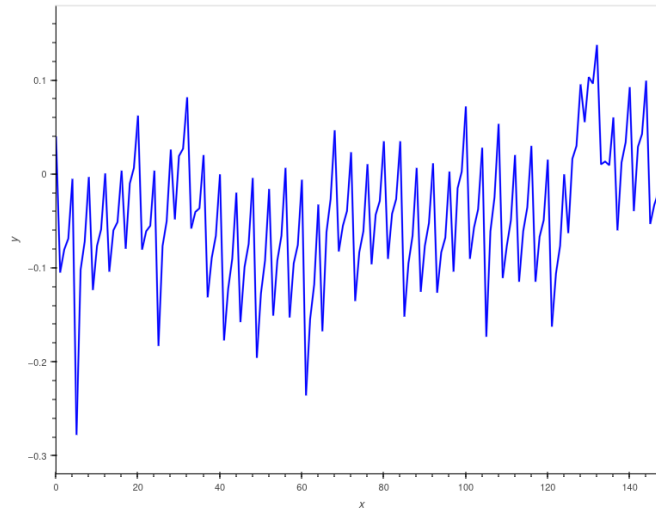


Fig. 19: Sending a password with a single char ‘h’

13. Next, we run the next cell, which sends a password with a single character ‘a’ to the target board and generates the plot of the collected power trace.

Sending a password with a single char ‘a’

```
trace_test2 = cap_pass_trace("a\n")

#Basic sanity check
assert(len(trace_test2) == 150)
print("✓ OK to continue!")

cw.plot(trace_test2).opts(color='green')
```

```
0
PASSWORD FAIL
****Safe-o-matic 3000 Booting...
Decrypting database..[DONE]
```

Fig. 20: Sending a password with a single char ‘a’

We should expect the program indicates that the password is incorrect and says ‘PASSWORD FAIL’ from the terminal. We should also obtain a power trace associated with the execution of the program given this password as shown in Fig. 21.

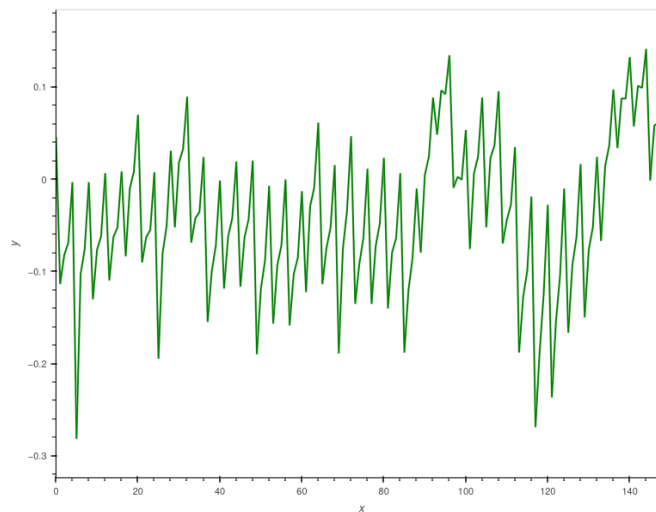


Fig. 21: Sending a password with a single char 'a'

At this point, *we should realize that the patterns of the two power traces from 'h' and 'a' are relatively different.* However, it is not clear which one is the correct first character.

We can repeat the process with other characters to figure it out.

14. Next, we run the next cell, which sends a password with a single character 'b' to the target board and generates the plot of the collected power trace.

Sending a password with a single char 'b'

```
trace_test3 = cap_pass_trace("b\n")

#Basic sanity check
assert(len(trace_test3) == 150)
print("✓ OK to continue!")

cw.plot(trace_test3).opts(color='red')
```

```
0
PASSWORD FAIL
****Safe-o-matic 3000 Booting...
Decrypting database..[DONE]
```

Fig. 22: Sending a password with a single char 'b'

We should expect the program indicates that the password is incorrect and says 'PASSWORD FAIL' from the terminal. We should also obtain a power trace associated with the execution of the program given this password as shown in Fig. 23.

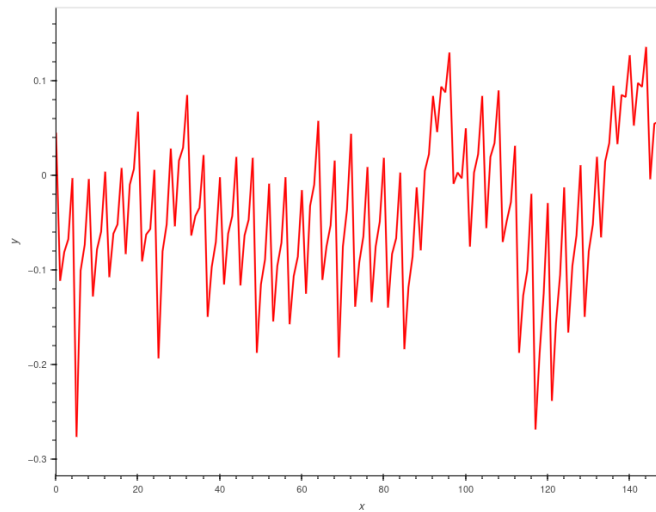


Fig. 23: Sending a password with a single char ‘b’

If we compare carefully, it is relatively easy to see that the power traces from ‘a’ and ‘b’ are almost the same while the one from ‘h’ is different. It feels like ‘h’ could be the correct first character in the hard-coded password. Perhaps we can try one more to further verify!

15. Next, we run the next cell, which sends a password with a single character ‘c’ to the target board and generates the plot of the collected power trace.

Sending a password with a single char ‘c’

```
trace_test4 = cap_pass_trace("c\n")

#Basic sanity check
assert(len(trace_test4) == 150)
print("✓ OK to continue!")

cw.plot(trace_test4).opts(color='orange')
```

```
0
PASSWORD FAIL
*****Safe-o-matic 3000 Booting...
Decrypting database..[DONE]
```

Fig. 24: Sending a password with a single char ‘c’

We should expect the program indicates that the password is incorrect and says ‘PASSWORD FAIL’ from the terminal. We should also obtain a power trace associated with the execution of the program given this password as shown in Fig. 25.

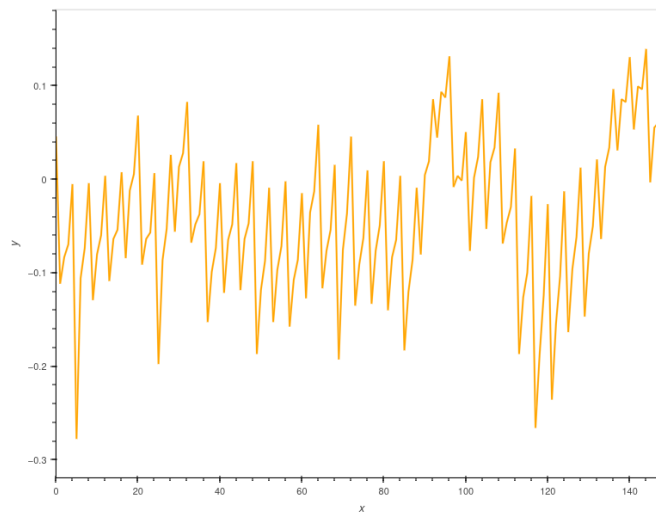


Fig. 25: Sending a password with a single char 'c'

Now, we can see that the power trace from 'c' is almost the same as the ones from 'a' and 'b' but is different from the one derived by 'h'. In other words, the one from 'h' is distinguishable from the other three, although the program on the board says that everyone of them is incorrect password as a whole.

16. We can further verify this by running the next cell, which plots all the four traces together.

Sending a password with a single char 'c'

```
trace_test4 = cap_pass_trace("c\n")

#Basic sanity check
assert(len(trace_test4) == 150)
print("✓ OK to continue!")

cw.plot(trace_test4).opts(color='orange')
```

```
0
PASSWORD FAIL
*****Safe-o-matic 3000 Booting...
Decrypting database..[DONE]
```

Fig. 26: Sending a password with a single char 'c'

And we can observe the plot as below, where the blue one is generated from 'h'. The green and red traces from 'a' and 'b' are mostly overwritten by the orange one from 'c' but we can still see some parts of these two.

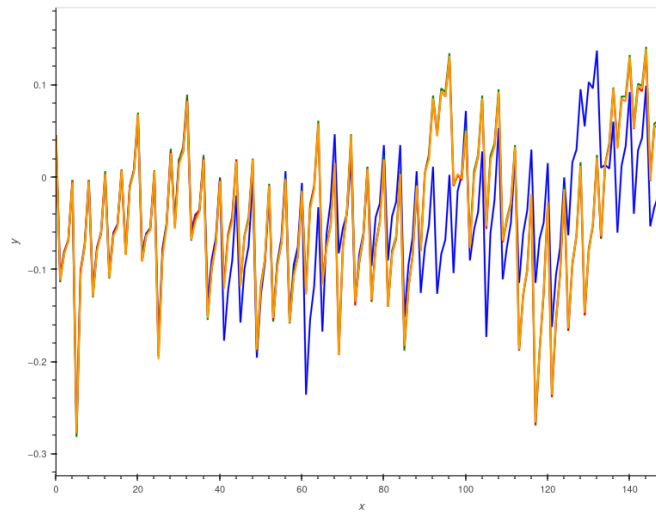


Fig. 27: Plotting four traces together to identify the distinguishable one

Simple Power Analysis Result. With this simple power analysis above, we can confirm that ‘h’ is actually the first character of the password hard-coded in the program. We can use the same approach to figure out the remaining characters in the password one by one.

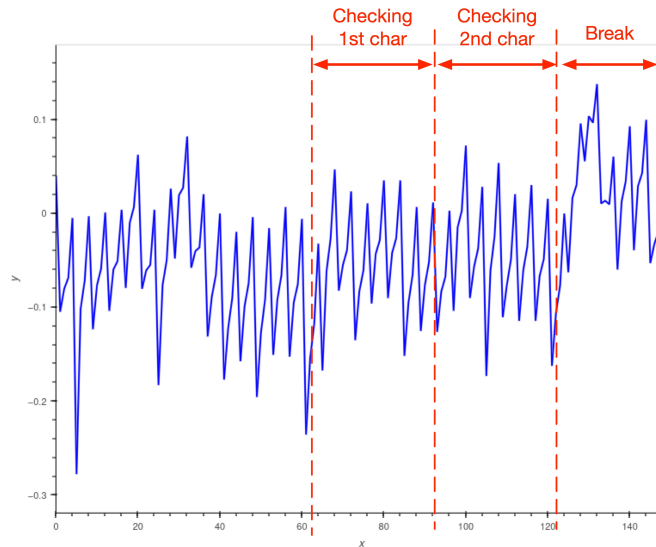


Fig. 28: Power trace breakdown

Additional Breakdown: If we break down the power traces a bit further by comparing the one from ‘h’ and other ones, we can also infer as below, where the verification of the first char and second char share repeatable pattern (as they are the same operation/instructions). Then, the high peak towards the end is associated with the instructions from breaking the program when the second character is not correct. In other words, we should expect the similar power pattern will

from the part ‘Checking 2nd char’ will happen again for the 3rd char before the ‘break’ when we guess the 2nd character correctly.

5 Conclusion

This concludes this tutorial.